

NAME : P SAI VENKAT

ROLL NO : 2403A510G0

SUBJECT : AI ASSISTED CODING

ASSIGNMENT : 10.4

Task 1: Syntax and Error Detection

Task: Identify and fix syntax, indentation, and variable errors in the given script.

```
# buggy_code_task1.py
def add_numbers(a, b)
result = a + b
return reslt
print(add_numbers(10 20))
```

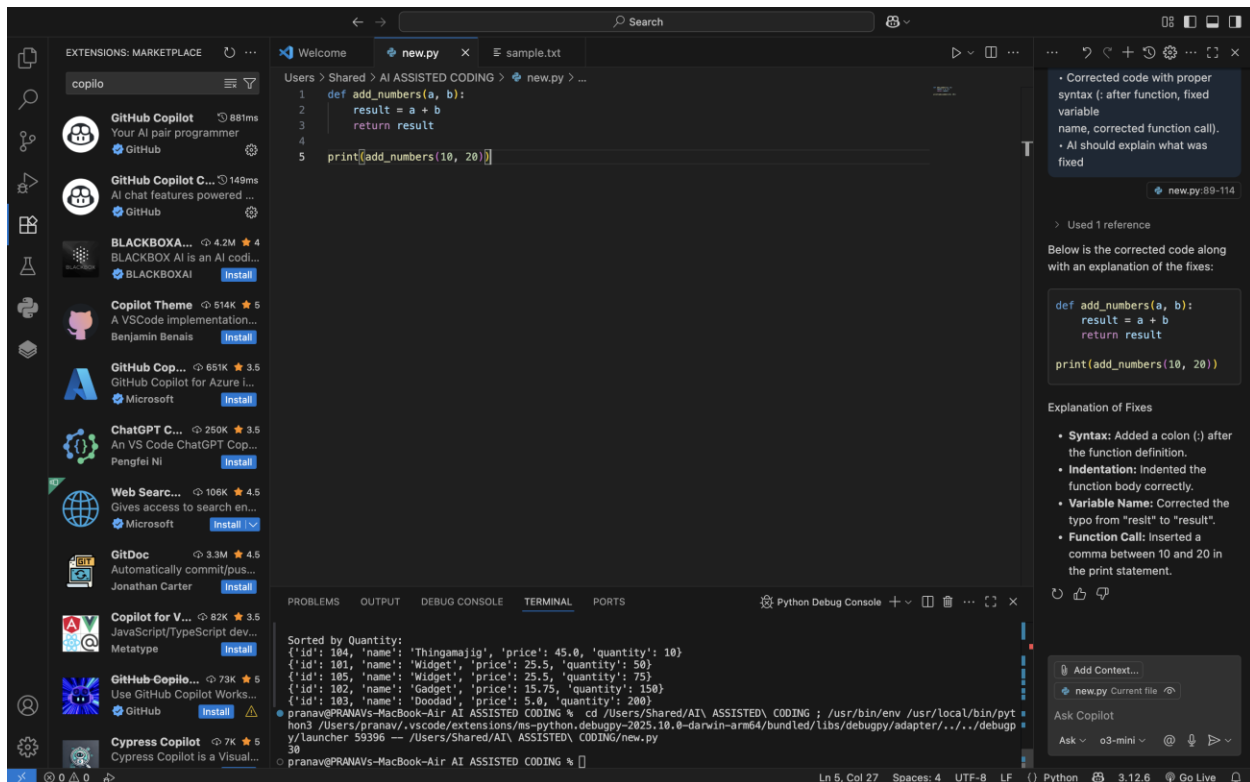
Expected Output:

- Corrected code with proper syntax (: after function, fixed variable name, corrected function call).
- AI should explain what was fixed.

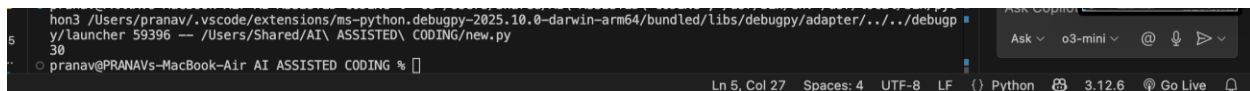
PROMPT USED :

To identify and fix syntax, indentation, and variable errors in the given script

CODE GENERATED :



OUTPUT :



Task 2: Logical and Performance Issue Review

Task: Optimize inefficient logic while keeping the result correct.

buggy_code_task2.py

```
def find_duplicates(nums):
```

```
    duplicates = []
```

```
    for i in range(len(nums)):
```

```
        for j in range(len(nums)):
```

```
            if i != j and nums[i] == nums[j] and nums[i] not in duplicates:
```

```
                duplicates.append(nums[i])
```

```
    return duplicates
```

```
numbers = [1,2,3,2,4,5,1,6,1,2]
```

```
print(find_duplicates(numbers))
```

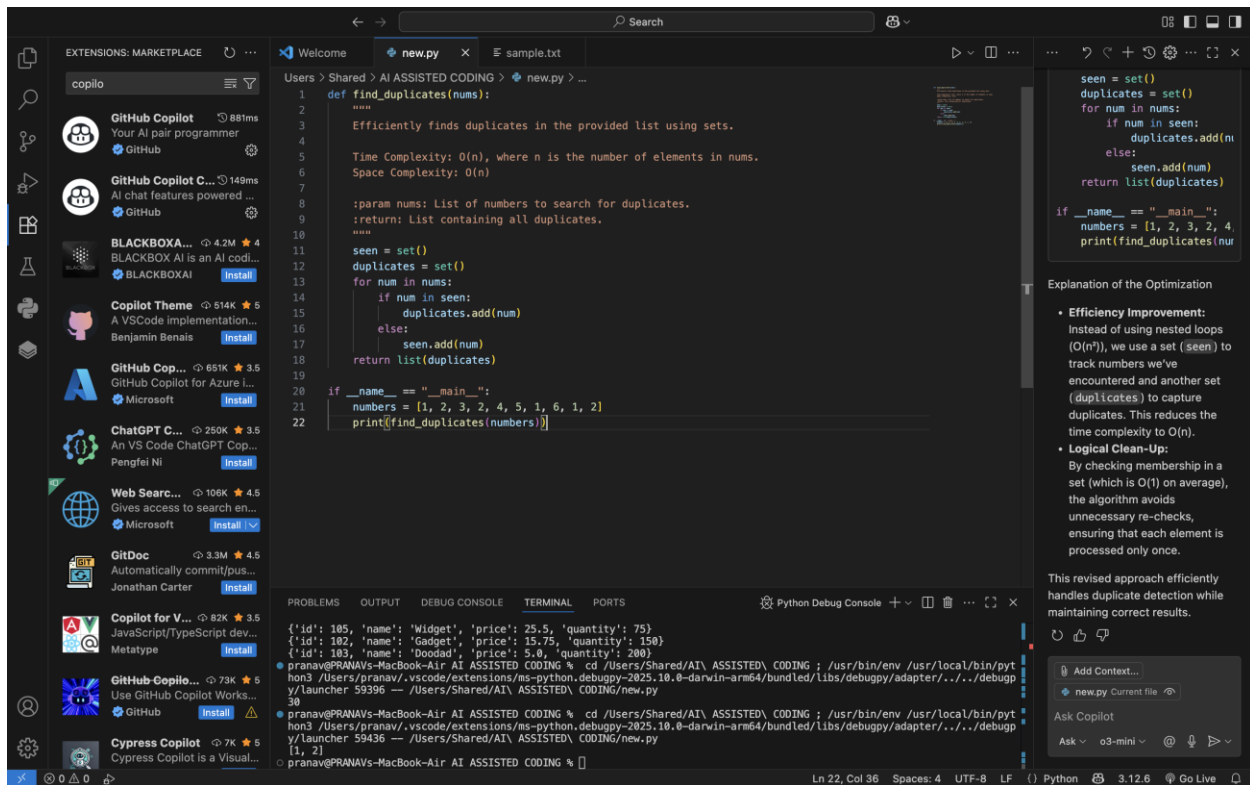
Expected Output:

- More efficient duplicate detection (e.g., using sets).
- AI should explain the optimization.

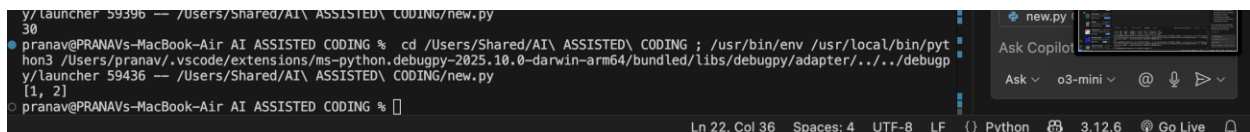
PROMPT USED :

To optimize inefficient logic while keeping the result correct for the given code

CODE GENERATED :



OUTPUT :



Task 3: Code Refactoring for Readability

Task: Refactor messy code into clean, PEP 8-compliant, well-structured code.

buggy_code_task3.py

```
def c(n):
x=1
for i in range(1,n+1):
x=x*i
return x
print(c(5))
```

Expected Output:

Function renamed to calculate_factorial.

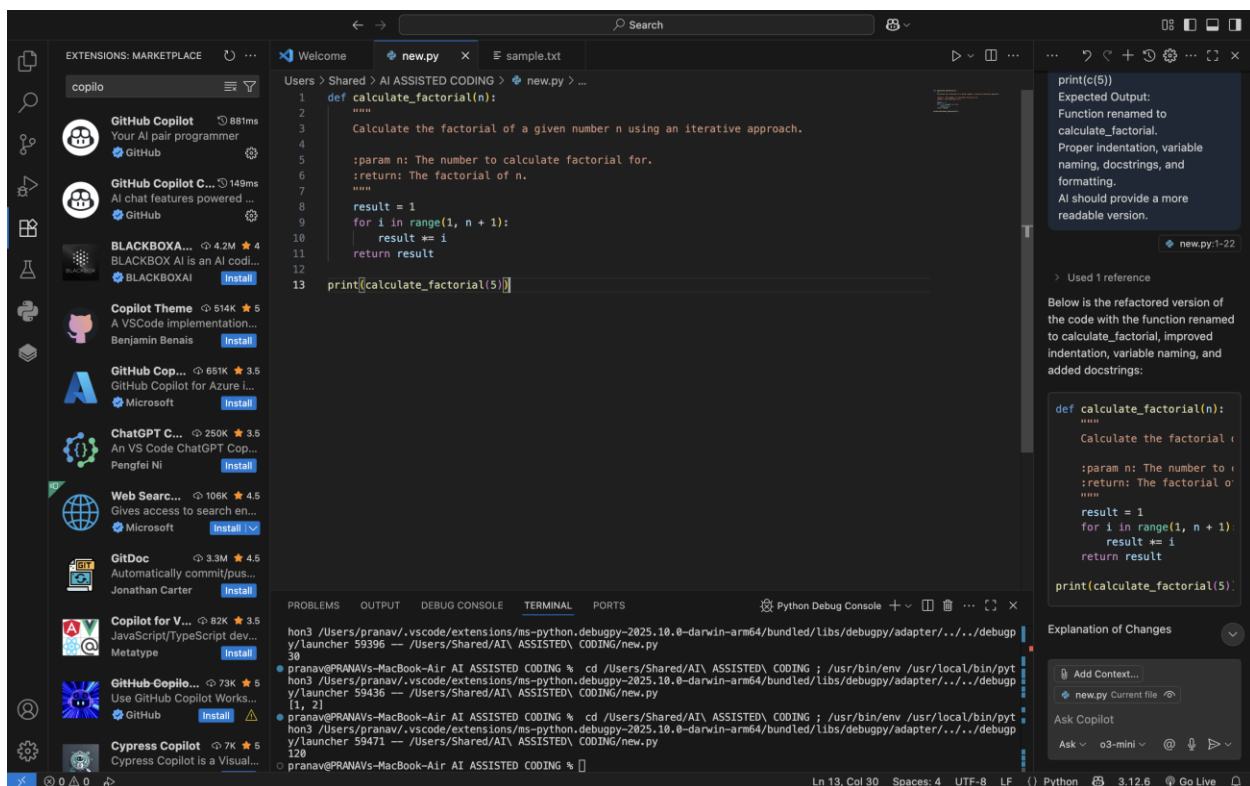
Proper indentation, variable naming, docstrings, and formatting.

AI should provide a more readable version.

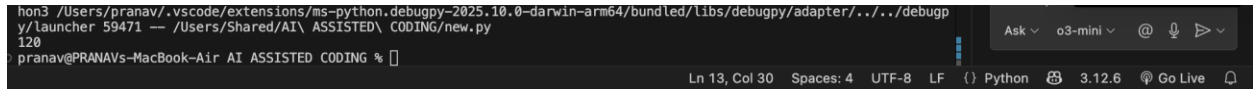
PROMPT USED :

To Refactor messy code into clean, PEP 8-compliant, well-structured code.

CODE GENERATED :



OUTPUT :



```
hon3 /Users/pranav/.vscode/extensions/ms-python.debugpy-2025.10.0-darwin-arm64/bundled/libs/debugpy/adapter/../../debugpy/launcher 59471 -- /Users/Shared/AI\ ASSISTED\ CODING/new.py
120
pranav@PRANAVs-MacBook-Air AI ASSISTED CODING %
```

Task 4: Security and Error Handling Enhancement

Task: Add security practices and exception handling to the code.

buggy_code_task4.py

```
import sqlite3
```

```
def get_user_data(user_id):
```

```
    conn = sqlite3.connect("users.db")
```

```
    cursor = conn.cursor()
```

```
    query = f"SELECT * FROM users WHERE id = {user_id};" #
```

Potential SQL injection risk

```
    cursor.execute(query)
```

```
    result = cursor.fetchall()
```

```
    conn.close()
```

```
    return result
```

```
user_input = input("Enter user ID: ")
```

```
print(get_user_data(user_input))
```

Expected Output:

Safe query using parameterized SQL (? placeholders).

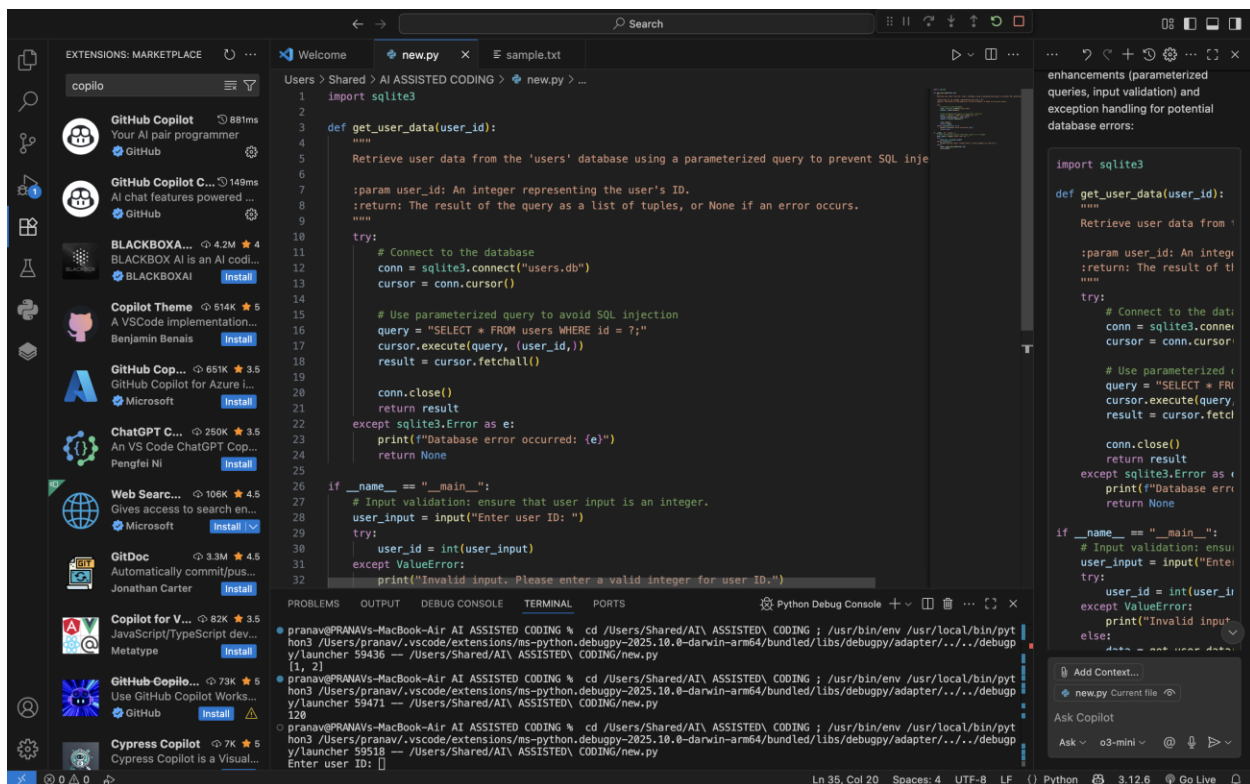
Try-except block for database errors.

Input validation before query execution

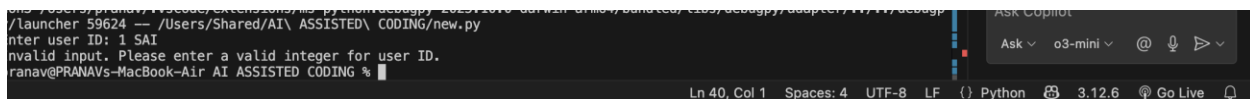
PROMPT USED :

To Add security practices and exception handling to the given code

CODE GENERATED :



OUTPUT GENERATED :



Task 5: Automated Code Review Report Generation

Task: Generate a review report for this messy code.

buggy_code_task5.py

```
def calc(x,y,z):
if z=="add":
return x+y
elif z=="sub": return x-y
elif z=="mul":
return x*y
elif z=="div":
return x/y
else: print("wrong")
print(calc(10,5,"add"))
print(calc(10,0,"div"))
```

Expected Output:

AI-generated review report should mention:

- o Missing docstrings
- o Inconsistent formatting (indentation, inline return)
- o Missing error handling for division by zero
- o Non-descriptive function/variable names
- o Suggestions for readability and PEP 8 compliance

PROMPT USED :

To Generate a review report for this messy code.

Issues identified:

Missing Docstrings:

No documentation is provided for the function. Adding a docstring would help other developers understand its purpose, parameters, and return values.

Inconsistent Formatting:

Indentation is inconsistent (e.g., some branches are on the same line as the elif statement while others are not).

Return statements should be consistently placed on their own line for better readability.

Lack of Error Handling:

Division by zero is not handled. If y is zero and z is "div", the function will raise a runtime error.

Instead of printing "wrong" in the default case, it might be better to raise a meaningful exception.

Non-Descriptive Names:

The function name calc and parameter names x, y, and z are not descriptive.

Consider renaming the function (e.g., calculate_operation) and parameters (e.g., num1, num2, operation) to improve clarity.

PEP 8 Compliance Suggestions:

Use proper spacing around operators and after commas.

Use consistent line breaks and avoid inline conditions where it reduces clarity.

Consider separating concerns: validating input, performing calculations, and error handling.

Recommendations for Improvement:

Rename the function and variables to be more descriptive.

Add a docstring to explain what the function does and describe its parameters and return value.

Refactor the conditional statements for consistent formatting.

Add error handling for division by zero (e.g., using a try-except block or checking before performing division).

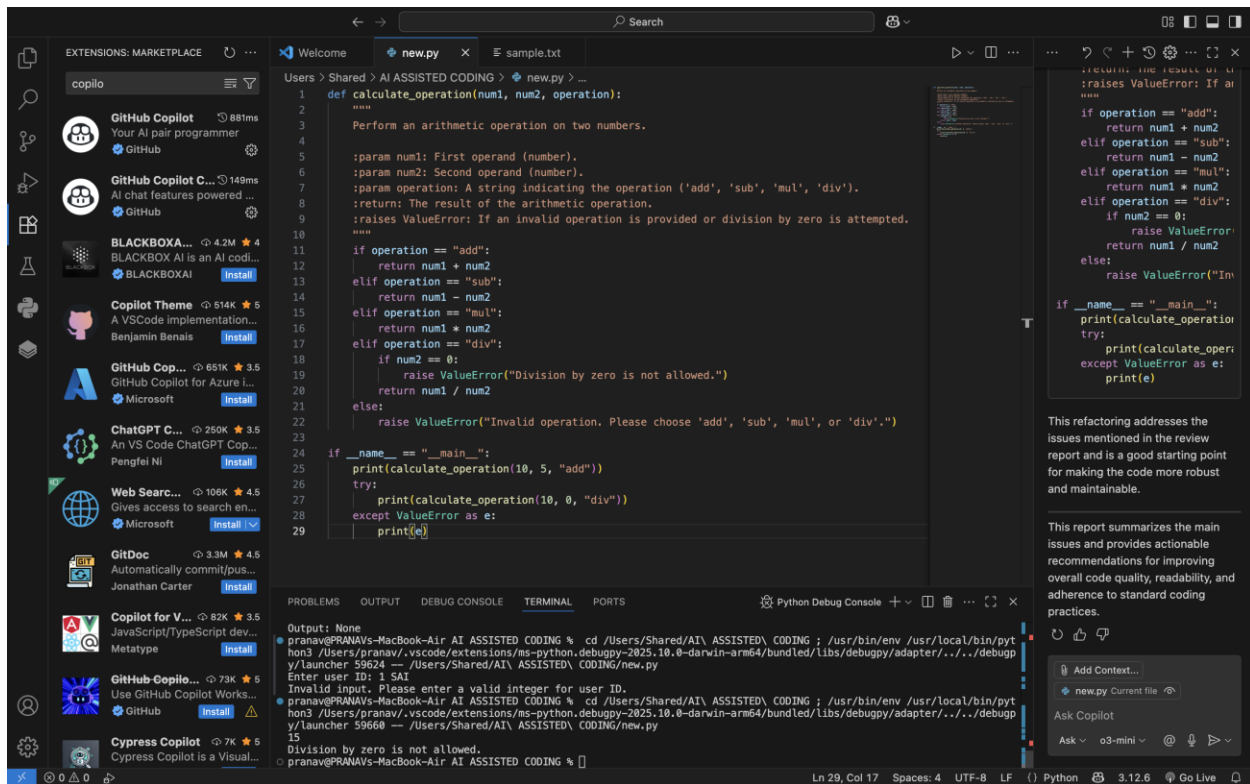
Return/raise meaningful errors when encountering invalid operations rather than merely printing an error message.

Review and format the code according to PEP 8 style guidelines.

Example Refactoring Idea

While not required in the review report, an improved version of the function might look like this:

CODE GENERATED :



OUTPUT:

